

EECE 7205: Fundamentals of Computer
Engineering
Assignment 1

Hamza Ghosheh

September 24, 2026

Contents

1	Problem 1: Merge Intervals	4
1.1	Understand the Problem	4
1.2	Design the Solution	4
1.3	Pseudocode	5
1.4	Correctness Explanation	5
1.5	Complexity Analysis	6
1.6	C++ Implementation	6
1.7	Testing	9
2	Problem 2: Sort 0s, 1s, and 2s	11
2.1	Understand the Problem	11
2.2	Design the Solution	11
2.3	Pseudocode	11
2.4	Correctness Explanation	12
2.5	Complexity Analysis	12
2.6	C++ Implementation	12
2.7	Testing	13
3	Problem 3: Largest Number	15
3.1	Understand the Problem	15
3.2	Design the Solution	15
3.3	Pseudocode	15
3.4	Correctness Explanation	16
3.5	Complexity Analysis	16
3.6	C++ Implementation	16
3.7	Testing	17
4	Problem 4: Group Anagrams	18
4.1	Understand the Problem	18
4.2	Design the Solution	18
4.3	Pseudocode	19
4.4	Correctness Explanation	19
4.5	Complexity Analysis	19
4.6	C++ Implementation	20
4.7	Testing	20
5	Problem 5: Meeting Intervals	22
5.1	Understand the Problem	22
5.2	Design the Solution	22
5.3	Pseudocode	23
5.4	Correctness Explanation	23
5.5	Complexity Analysis	24
5.6	C++ Implementation	24
5.7	Testing	26

6	Problem 6: K Closest Points to the Origin	28
6.1	Understand the Problem	28
6.2	Design the Solution	28
6.3	Pseudocode	29
6.4	Correctness Explanation	29
6.5	Complexity Analysis	30
6.6	C++ Implementation	30
6.7	Testing	32
7	Problem 7: K Most Frequent Values	34
7.1	Understand the Problem	34
7.2	Design the Solution	34
7.3	Pseudocode	35
7.4	Correctness Explanation	35
7.5	Complexity Analysis	35
7.6	C++ Implementation	36
7.7	Testing	37
8	Problem 8: Merge K Sorted Arrays	39
8.1	Understand the Problem	39
8.2	Design the Solution	39
8.3	Pseudocode	40
8.4	Correctness Explanation	40
8.5	Complexity Analysis	40
8.6	C++ Implementation	41
8.7	Testing	42
9	Problem 9: Kth Largest Element	44
9.1	Understand the Problem	44
9.2	Design the Solution	44
9.3	Pseudocode	44
9.4	Correctness Explanation	44
9.5	Complexity Analysis	45
9.6	C++ Implementation	45
9.7	Testing	45
10	Problem 10: Count Inversions	47
10.1	Understand the Problem	47
10.2	Design the Solution	47
10.3	Pseudocode	48
10.4	Correctness Explanation	48
10.5	Complexity Analysis	49
10.6	C++ Implementation	49
10.7	Testing	50
11	References	52

1 Problem 1: Merge Intervals

1.1 Understand the Problem

The goal with this problem is to return non-overlapping interval(s). The input is an array of intervals, where each interval has two integers, and the output is a simplified set of intervals that are non-overlapping. We want to generate a program that can help simplify it for us if such cases do take place. Moreover, it is important to note that empty inputs, single intervals, overlapping intervals, and nested intervals are all significant edge cases to keep in mind. For simplification purposes, we must also assume that the first integer is less than or equal to the second. Furthermore, some constraints to make this problem solvable would be that each interval is constrained by two integers, which symbolize beginning to end.

1.2 Design the Solution

My approach to designing a solution for this would be to sort through the arrays of intervals provided in the input one at a time and compare overlapping with each array to see if any overlaps occur. If there is overlapping, merging would take place. If not, we continue until we reach the end. Sorting the intervals from lowest to highest makes it easier for both the user and the program to help identify which intervals overlap.

1.3 Pseudocode

Algorithm 1 Merge Intervals

```
1: function MERGEINTERVALS(intervals)
2:   if intervals is empty then
3:     return zero
4:   end if
5: end function
6: function SORTING(intervals)
7:   sort intervals by starting value
8:   merged  $\leftarrow$  empty list
9:   current  $\leftarrow$  first interval
10:  for each next interval starting from the second interval do
11:    if next.start  $\leq$  current.end then
12:      current.end  $\leftarrow$  max(current.end, next.end)
13:    else
14:      add current to merged
15:      Call the mergeSort function
16:      current  $\leftarrow$  next
17:    end if
18:  end for
19: end function
20: function MERGESORT(intervals)
21:   if left value is less than right value then
22:     moves to left interval position
23:     moves to right interval position
24:   else
25:     add current to merged
26:     return merged
27:   end if
28: end function
```

1.4 Correctness Explanation

This algorithm works because comparing the second integer in the current interval with the first integer in the next interval is important to note if the first interval did not end. If the starting value of the second interval is greater than the ending value of the first, there is no overlap and thus the program proceeds to the next array. On the other hand, if the starting value of the second interval is less than the ending value of the first, overlap takes place and merging takes place accordingly. Furthermore, the function also handles a nested interval to disregard any interval that is enclosed with a larger one. Lastly, if the input is empty, the return is an empty set.

1.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log n)$

Average Case: $O(n \log n)$

Best Case: $O(n \log n)$

Space: $O(n)$

Scanning through intervals: $O(n)$

1.6 C++ Implementation

```
void mergeSort(int intervals[][2], int leftValue, int
               rightValue);

void sorting(int intervals[][2], int leftValue, int middle,
             int rightValue);

int mergeIntervals(int intervals[][2], int size) {
    if (size == 0) {
        return 0;
    }

    mergeSort(intervals, 0, size - 1);

    int (*merged)[2] = new int[size][2];

    int mergedSize = 0;

    int currentStart = intervals[0][0];
    int currentEnd = intervals[0][1];

    for (int i = 1; i < size; i++) {
        if (intervals[i][0] <= currentEnd) {
            currentEnd = max(currentEnd, intervals[i][1]);
        }
        else {
            merged[mergedSize][0] = currentStart;
            merged[mergedSize][1] = currentEnd;
            mergedSize++;

            currentStart = intervals[i][0];
            currentEnd = intervals[i][1];
        }
    }

    merged[mergedSize][0] = currentStart;
    merged[mergedSize][1] = currentEnd;
    mergedSize++;
}
```

```

        for (int i = 0; i < mergedSize; i++) {
            intervals[i][0] = merged[i][0];
            intervals[i][1] = merged[i][1];
        }

        delete[] merged;
        return mergedSize;
    }

void sorting(int intervals[][2], int leftValue,
            int middle, int rightValue) {

    int x = 0;
    int y = 0;
    int k = leftValue;

    if (leftValue < rightValue) {
        int leftSize = middle - leftValue + 1;
        int rightSize = rightValue - middle;

        int (*leftInterval)[2] = new int[leftSize][2];
        int (*rightInterval)[2] = new int[rightSize][2];

        for (int i = 0; i < leftSize; i++) {
            leftInterval[i][0] = intervals[leftValue + i][0];
            leftInterval[i][1] = intervals[leftValue + i][1];
        }

        for (int j = 0; j < rightSize; j++) {
            rightInterval[j][0] = intervals[middle + 1 + j][0];
            rightInterval[j][1] = intervals[middle + 1 + j][1];
        }

        while (x < leftSize && y < rightSize) {
            if (leftInterval[x][0] <= rightInterval[y][0]) {
                intervals[k][0] = leftInterval[x][0];
                intervals[k][1] = leftInterval[x][1];
                x++;
                k++;
            }
            else {
                intervals[k][0] = rightInterval[y][0];
                intervals[k][1] = rightInterval[y][1];
                y++;
                k++;
            }
        }
    }
}

```

```

        }
    }

    while (x < leftSize) {
        intervals[k][0] = leftInterval[x][0];
        intervals[k][1] = leftInterval[x][1];
        x++;
        k++;
    }

    while (y < rightSize) {
        intervals[k][0] = rightInterval[y][0];
        intervals[k][1] = rightInterval[y][1];
        y++;
        k++;
    }

    delete[] leftInterval;
    delete[] rightInterval;
}

}

void mergeSort(int intervals[][2],
               int leftValue, int rightValue) {

    if (leftValue < rightValue) {
        int middle = leftValue +
                      (rightValue - leftValue) / 2;

        mergeSort(intervals, leftValue, middle);
        mergeSort(intervals, middle + 1, rightValue);
        sorting(intervals, leftValue, middle, rightValue);
    }
}

void printIntervals(int intervals[][2], int size) {
    for (int i = 0; i < size; i++) {
        cout << "[" << intervals[i][0]
              << ", " << intervals[i][1] << "]" ";
    }

    cout << endl;
}

```


1.7 Testing

```
Test Case 1 (Overlapping intervals):  
Input: [1, 12] [3, 7] [10, 89]  
Output: [1, 89]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205
```

Figure 1: Problem 1 - Test Case 1: Overlapping Intervals

```
Test Case 2 (Nested intervals):  
Input: [31, 37] [22, 44]  
Output: [22, 44]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 2: Problem 1 - Test Case 2: Nested Intervals

```
Test Case 3 (Single Interval):  
Input: [3, 12]  
Output: [3, 12]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 3: Problem 1 - Test Case 3: Single Interval

```
Test Case 4 (Mix of Overlap and Non-Overlap):  
Input: [3, 66] [5, 21] [55, 80]  
Output: [3, 80]  
hamzaghosseh@Hamzas-MacBook-Air EECE7205 %
```

Figure 4: Problem 1 - Test Case 4: Mix of Overlapping and non-Overlapping Intervals

```
Test Case 5 (Empty Interval):  
Input: []  
Output: []
```

Figure 5: Problem 1 - Test Case 5: Empty Intervals

2 Problem 2: Sort 0s, 1s, and 2s

2.1 Understand the Problem

The goal with this problem is to take an input filled with 0s, 1s, and 2s and output a an array in ascending order. The final solution should be achieved without having to use the `std::sort()` function that automatically takes care of it. The edge cases used to test this algorithm include an empty array, an array in ascending order, and an array that is jumbled up randomly. Let us assume that the array only contains integers of 0, 1, and 2.

2.2 Design the Solution

The design itself will use the Dutch National Flag algorithm, which utilizes three pointers *low*, *mid*, and *high*. Essentially, the *low* and *mid* pointers will be indexed at the smallest position in the array, which is always 0, not to be confused with the lowest value in the array. Meanwhile the *high* is situated at the highest value in the array at $n - 1$. Once run, the program uses *mid* to assess each value. If the value is 0, it sends it at the beginning of the array, and if the value is 1 it is moved towards the middle, and if the value is 2 it pushes it towards the end. The process repeats until everything is arranged in ascending order.

2.3 Pseudocode

Algorithm 2 Sort 0s, 1s, and 2s

```
1: function DUTCHALGORITHM(array, size)
2:   initialize low  $\leftarrow$  0
3:   initialize mid  $\leftarrow$  0
4:   initialize high  $\leftarrow$  size - 1
5:   while mid  $\leq$  high do
6:     if array[mid] = 0 then
7:       swap low and mid values
8:       increment low with addition
9:       increment mid with addition
10:    else if array[mid] = 1 then
11:      increment mid with addition
12:    else
13:      swap mid and high values
14:      decrement high with subtraction
15:    end if
16:  end while
17: end function
```

2.4 Correctness Explanation

The algorithm is correct since each value, when encountered, is properly fitted in its correct place. When a 0 is encountered, it is swapped toward the beginning of the array using the low pointer. When a 1 is encountered, it is left in the middle and the mid pointer moves forward. When a 2 is encountered, it is swapped toward the end of the array using the high pointer. Once mid passes high, the algorithm stops, and at this point, all 0s are located before the 1s, and all 1s are located before the 2s, hence returning an array sorted in ascending order.

2.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n)$

Average Case: $O(n)$

Best Case: $O(n)$

Space: $O(1)$

Scanning through the array: $O(n)$

2.6 C++ Implementation

```
void dutchAlgorithm(int array[], int size) {
    int lowVal = 0;
    int midVal = 0;
    int highVal = size - 1;

    while (midVal <= highVal) {
        if (array[midVal] == 0) {
            swap(array[lowVal], array[midVal]);
            lowVal++;
            midVal++;
        }
        else if (array[midVal] == 1) {
            midVal++;
        }
        else {
            swap(array[midVal], array[highVal]);
            highVal--;
        }
    }
}

void arrayDisplay(int array[], int size) {
    for (int i = 0; i < size; i++) {
        cout << array[i] << " ";
    }
}
```

```
    cout << endl;  
}
```

2.7 Testing

```
Test Case 1 (Jumbled-up Array):  
Input: 0 1 0 2 2 0 1 2  
Output: 0 0 0 1 1 2 2 2  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 6: Problem 2 - Test Case 1: Jumbled Up Array

```
Test Case 2 (Array in Ascending Order):  
Input: 0 0 1 1 1 2 2 2  
Output: 0 0 1 1 1 2 2 2  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 7: Problem 2 - Test Case 2: Array in Ascending Order

```
Test Case 3 (Array in Descending Order):  
Input: 2 2 1 1 1 0 0 0  
Output: 0 0 0 1 1 1 2 2  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 8: Problem 2 - Test Case 3: Array in Descending Order

```
Test Case 4 (All the same values in the Array):  
Input: 2 2 2 2 2 2 2 2  
Output: 2 2 2 2 2 2 2 2  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 9: Problem 2 - Test Case 4: All the Same Values in the Array

```
Test Case 5 (Empty Array):  
Input: []  
Output: []  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 10: Problem 2 - Test Case 5: Empty Array

3 Problem 3: Largest Number

3.1 Understand the Problem

The goal of this program here is to take an array of n integers as an input, rearrange each integer in the array to create the largest possible number, and return that value as a string for the output. Important edge cases to keep in mind are empty arrays, single digit values, values with the same starting integer, and an array with all zeros. Assume that every value in the array is a non-negative integer and that two or more zeros as an integer is impossible.

3.2 Design the Solution

The algorithm itself will assess which values in the array are structured in an order that will produce the largest possible number. Two values will be compared by concatenating them in both possible orders to see which one produces the larger number. Given two values x and y , the function will compare the concatenation of xy with yx to see which one is larger. This process repeats until we have a structured array in the desired order, and once everything is concatenated into one integer it returns as a string.

3.3 Pseudocode

Algorithm 3 Largest Number

```
1: function BOOLEAN COMPARING(string x, string y)
2:   return  $x + y$  is greater than  $y + x$ 
3: end function
4: function CONSTRUCTLARGESTINT(array, size)
5:   initialize vector myString
6:   Utilize a to string
7:   Comparing two strings  $a$  and  $b$ :
8:   Place  $a$  before  $b$  if  $(a + b) > (b + a)$ 
9:   if the myString is "0" then
10:    return "0"
11:  end if
12:  SORT(myString.begin, myString.end, comparing())
13:  completeString = completeString + 1
14:  completeString results in empty string
15:  for each string in the sorted array do
16:     $completeString \leftarrow completeString + string$ 
17:  end for
18:  return completeString
19: end function
```

3.4 Correctness Explanation

This function is correct because it factors in all possibilities of concatenating two strings together via comparison, and covers all possible edge cases to ensure there are no special exceptions.

3.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log n)$

Average Case: $O(n \log n)$

Best Case: $O(n \log n)$

Space: $O(n)$

Scanning through the array: $O(n)$

3.6 C++ Implementation

```
bool comparing(string x, string y) {
    return x + y > y + x;
}

string constructLargestInt(vector<int> integerVect) {

    if (integerVect.size() == 0) {
        return "";
    }

    vector<string> myString;

    for (int i = 0; i < integerVect.size(); i++) {
        myString.push_back(to_string(integerVect[i]));
    }

    sort(myString.begin(), myString.end(), comparing);

    if (myString[0] == "0") {
        return "0";
    }

    string completeString = "";

    for (int i = 0; i < myString.size(); i++) {
        completeString += myString[i];
    }

    return completeString;
}
```


3.7 Testing

```
Test Case 1 (All Single Digits):  
Input: [4, 5, 7, 9]  
Output: 9754  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 11: Problem 3 - Test Case 1: All Single Digits

```
Test Case 2 (All Double Digits):  
Input: [24, 45, 22, 52]  
Output: 52452422  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 12: Problem 3 - Test Case 2: All Double Digits

```
Test Case 3 (All values share same first digit):  
Input: [24, 251, 29, 220]  
Output: 2925124220  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 13: Problem 3 - Test Case 3: All Values share the Same First Digit

```
Test Case 4 (All Values are Equal):  
Input: [12, 12, 12, 12]  
Output: 12121212  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 14: Problem 3 - Test Case 4: All Values are Equal

```
Test Case 5 (All Values are zeros):  
Input: [0, 0, 0, 0]  
Output: 0  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 15: Problem 3 - Test Case 5: All Values are Zero

4 Problem 4: Group Anagrams

4.1 Understand the Problem

The goal of this problem is to group words that are anagrams of each other into arrays. The input is an array of words (strings), and the output will be an array of arrays grouping any words that are anagrams of each other into a sub-array (strings). Any word that has no anagram of itself in the input will be in its own array when the output is displayed. Edge cases for this function would be an empty array, strings of matching anagrams, strings of non-matching anagrams, a mixture of both, and any duplicate words of one another. Important assumptions to keep in mind include that every string must be lowercase and that words in each group should retain their original order in the array.

4.2 Design the Solution

The function of design will be used to check each individual character in each word. We would basically create a key for all strings by looking at the characters in each string and sorting them. Any string that has a matching key will be considered an anagram. As far as positioning goes, we would have to look at the index of each word and note the order of strings. The process would repeat until the array is finished, and once that happens, the function will return an array of arrays of all anagrams.

4.3 Pseudocode

Algorithm 4 Group Anagrams

```
1: function SORTANAGRAMS(vector initialVector)
2:   groups  $\leftarrow$  empty vector
3:   keys  $\leftarrow$  empty vector
4:   for  $i \leftarrow 0$  to initialVectorsize  $-1$  do
5:     key  $\leftarrow$  word
6:     Call sort function in key
7:     Use a boolean found  $\leftarrow$  false
8:     for  $j \leftarrow 0$  to number of keys  $-1$  do
9:       if key = keys[ $i$ ] then
10:        Add word to sets[ $i$ ]
11:        Use a found  $\leftarrow$  true
12:        break
13:       end if
14:     end for
15:     if found = false then
16:       Add key to keys
17:       vector string newGroup gets initialized
18:       Create a new set containing word
19:       Push back the new data to groups
20:     end if
21:   end for
22:   return groups
23: end function
```

4.4 Correctness Explanation

This function works because it creates a key for each word and eventually groups all words with the same key into an array. Because anagrams have the same letters, sorting them is easy because they will all have the same key. Hence why all words with anagrams will be put together, while words without anagrams will be put in a singular array.

4.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n^2)$

Average Case: $O(n^2)$

Best Case: $O(n)$

Space: $O(n)$

Scanning through the array: $O(n)$

4.6 C++ Implementation

```
vector<vector<string>> sortAnagrams(vector<string>
    initialVector) {
    vector<vector<string>> groups;
    vector<string> keys;

    if (initialVector.size() == 0) {
        return groups;
    }

    for (int i = 0; i < initialVector.size(); i++) {
        string word = initialVector[i];
        string key = word;

        sort(key.begin(), key.end());
        bool found = false;

        for (int j = 0; j < keys.size(); j++) {
            if (key == keys[j]) {
                groups[j].push_back(word);
                found = true;
                break;
            }
        }

        if (found == false) {
            keys.push_back(key);
            vector<string> newGroup;
            newGroup.push_back(word);
            groups.push_back(newGroup);
        }
    }

    return groups;
}
```

4.7 Testing

```
Test Case 1 (All Words are Anagrams):
Input: [rat, tar, art]
Output: [[rat, tar, art]]
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 16: Problem 4 - Test Case 1: All Words are Anagrams

```
Test Case 2 (Multiple Anagrams):  
Input: [rat, tar, art, ape, pea]  
Output: [[rat, tar, art], [ape, pea]]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 17: Problem 4 - Test Case 2: Multiple Anagrams

```
Test Case 3 (No Words are Anagrams):  
Input: [twelve, three, rock, special]  
Output: [[twelve], [three], [rock], [special]]
```

Figure 18: Problem 4 - Test Case 3: No Words are Anagrams

```
Test Case 4 (Duplicate Words):  
Input: [twelve, twelve, bonk, knob]  
Output: [[twelve, twelve], [bonk, knob]]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 19: Problem 4 - Test Case 4: Duplicate Words

```
Test Case 5 (Empty Array):  
Input: []  
Output: []  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 20: Problem 4 - Test Case 5: Empty Array

5 Problem 5: Meeting Intervals

5.1 Understand the Problem

The goal with this problem is to understand whether all meetings can occur without any time conflicts, such that a person is able to attend all of them. The input contains intervals for meeting times, both start and end, and the output will either be true or false depending if there are any overlaps with intervals. Some edge cases for this include an empty input, a singular time, back-to-back intervals, and overlapping intervals. Assume that every interval includes only positive integers.

5.2 Design the Solution

To start with the design, we must sort out the intervals in ascending order, where the lowest number at the beginning is at the front and the largest integer for the second value is at the end. After that, we would basically compare to see if there are any overlaps. If there are no overlaps or if the intervals are back to back, then the function returns true. If there is overlap, the function returns false.

5.3 Pseudocode

Algorithm 5 Attending all Meetings

```
1: function SORTINGINTERVAL(meetings)
2:   if left is less than right then
3:     return true
4:     for  $i$  is less than the lefthalf do
5:       Iterate through the first interval to current interval
6:     end for
7:     for  $j$  is less than the righthalf do
8:       Iterate through the second interval to current interval
9:     end for
10:  end if
11:  while first element is less than the length of the left and second element
    is less than the length on the right do
12:    Sort by starting time
13:  end while
14:  while first element is less than the length of the left do
15:    increment through firstarray
16:  end while
17:  while second element is less than the length on the right do
18:    increment through secondarray
19:  end while
20:  cleanup temporary arrays
21: end function
22: function CHECKOVERLAP(intervals, left, right)
23:   if left is less than the right then
24:     Compute midpoint
25:     Recursively call checkOverlap
26:     Call Sorting Intervals
27:   end if
28: end function
29: function OVERLAPPINGINTERVALS
30:   if size is less than 1 then
31:     return true
32:   end if
33: end function
```

5.4 Correctness Explanation

This function works because there is a set way of organizing meetings in a set way that makes it easier to evaluate. After sorting, the comparison is made to see if there is any overlap between the connecting meeting intervals. That is where the true/false returning comes in.

5.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log n)$

Average Case: $O(n \log n)$

Best Case: $O(n \log n)$

Space: $O(n)$

Scanning through the array: $O(n)$

5.6 C++ Implementation

```
void sortingInterval(int current_Interval[][2], int leftNum,
    int middleNum, int rightNum) {

    int a = 0;
    int b = 0;
    int c = leftNum;

    if (leftNum < rightNum) {
        int length_l = middleNum - leftNum + 1;
        int length_r = rightNum - middleNum;

        int (*first_interval)[2] = new int[length_l][2];
        int (*second_interval)[2] = new int[length_r][2];

        for (int i = 0; i < length_l; i++) {
            first_interval[i][0] =
                current_Interval[leftNum + i][0];
            first_interval[i][1] =
                current_Interval[leftNum + i][1];
        }

        for (int j = 0; j < length_r; j++) {
            second_interval[j][0] =
                current_Interval[middleNum + 1 + j][0];
            second_interval[j][1] =
                current_Interval[middleNum + 1 + j][1];
        }

        while (a < length_l && b < length_r) {
            if (first_interval[a][0] <= second_interval[b]
                [0]) {
                current_Interval[a][0] =
                    first_interval[a][0];
                current_Interval[c][1] =
                    first_interval[a][1];
                a++;
                c++;
            }
        }
    }
}
```



```

        else {
            current_Interval[c][0] =
                second_interval[b][0];
            current_Interval[c][1] =
                second_interval[b][1];

            b++;
            c++;
        }
    }

    while (a < length_l) {
        current_Interval[c][0] =
            first_interval[a][0];
        current_Interval[c][1] =
            first_interval[a][1];
        a++;
        c++;
    }

    while (b < length_r) {
        current_Interval[c][0] =
            second_interval[b][0];
        current_Interval[c][1] =
            second_interval[b][1];
        b++;
        c++;
    }

    delete[] first_interval;
    delete[] second_interval;
}

void checkOverlap(int currentIntervals[][2], int leftVal,
    int rightVal) {
    if (leftVal < rightVal) {
        int midVal = leftVal + (rightVal - leftVal) / 2;

        checkOverlap(currentIntervals, leftVal, midVal);
        checkOverlap(currentIntervals, midVal + 1, rightVal)
            ;

        sortingInterval(currentIntervals, leftVal, midVal,
            rightVal);
    }
}

bool overlappingIntervals(int meetingIntervals[][2], int size
) {

```

```

    if (size <= 1) {
        return true;
    }

    checkOverlap(meetingIntervals, 0, size - 1);
    for (int i = 0; i < size; i++) {
        if (meetingIntervals[i][0] <
            meetingIntervals[i - 1][1]) {
            return false;
        }
    }

    return true;
}

```

5.7 Testing

```

Test Case 1 (Single Interval):

Input: [3, 5]

Output: True. All meeting times attendable.
hamzaghosseh@Hamzas-MacBook-Air EECE7205 %

```

Figure 21: Problem 5 - Test Case 1: Single Interval

```

Test Case 2 (Non-Overlapping Intervals):

Input: [3, 5][7, 10][10, 12]

Output: True. All meeting times attendable.
hamzaghosseh@Hamzas-MacBook-Air EECE7205 %

```

Figure 22: Problem 5 - Test Case 2: Non-Overlapping Intervals

```
Test Case 3 (Overlapping Intervals):  
Input: [3, 8][4, 10][11, 12]  
Output: False. Cannot attend meetings.  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 23: Problem 5 - Test Case 3: Overlapping Intervals

```
Test Case 4 (Same End/Begin Times):  
Input: [1, 5][5, 12]  
Output: True. All meeting times attendable.  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 24: Problem 5 - Test Case 4: Same End/Begin Times

6 Problem 6: K Closest Points to the Origin

6.1 Understand the Problem

The objective of this problem is to take points on a standard xy coordinate system, and based on a certain integer key, the program will return the closest points to the origin. Important edge cases to keep in mind are no inputs, equidistant inputs, and differing inputs. Assume that every input is given as an integer, and an integer k stays as a positive integer and does not exceed the total number of points given in the array.

6.2 Design the Solution

Before any way of implementation takes place, the primary focus here is to mathematically solve for the distance between the origin and the point given using the distance formula. Once we have the distance for each coordinate, one approach to solving this problem is to sort out all the distances in order from least to greatest. Another approach to solving this is implementing a heap that maintains only the k closest points are subject to being output, while the rest are being disregarded. For simplicity purposes, going with the sorting method makes the most sense since it is a straightforward way of calculating, organization, and point selection.

6.3 Pseudocode

Algorithm 6 Closest Points to Origin

```
1: function DISTANCEFORMULA( $x, y$ )
2:   return  $x * x + y * y$ 
3: end function
4: function DETERMINEPOINTDISTANCE(points,  $k$ )
5:   for  $i$  is less than  $leftSize$  do
6:   end for
7:   for  $j$  is less than  $rightSize$  do
8:   end for
9:   while both  $i$  is less than  $leftSize$  and  $j$  is less than  $rightSize$  do
10:    if  $initialDistance \leq comparisonDistance$  then
11:      increment  $i$ 
12:    else
13:      increment  $j$ 
14:    end if
15:  end while
16:  while  $i$  is less than  $leftSize$  do
17:    increment through coordinate set
18:  end while
19:  while  $j$  is less than  $rightSize$  do
20:    increment through coordinate set
21:  end while
22: end function
23: function SORTPOINTS(coordinates, left, right)
24:   if leftdistance is less than right distance then
25:     implement merge sort behavior using midpoint
26:     recursively call sortPoints
27:     call determinePointDistance
28:   end if
29: end function
30: function KCLOSESTPOINTS(points,  $k$ )
31:   if size equals zero then
32:     return emptyarray
33:   end if
34:   for  $i \leftarrow 0$  to  $k - 1$  do
35:     Add  $points[i]$  to result
36:   end for
37: end function
```

6.4 Correctness Explanation

This method is correct because it takes in all the parameters to evaluate the distance mathematically, while keeping in mind that the first k values in the list

closest to the origin will serve as the output. Returning the first k points will showcase the algorithm's success.

6.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log n)$

Average Case: $O(n \log n)$

Best Case: $O(n \log n)$

Space: $O(n)$

Scanning through the array: $O(n)$

6.6 C++ Implementation

```
int distanceFormula(int x, int y) {
    return x*x + y*y;
}

void determinePointDistance(int coordinateSet[][2], int
    X_coord, int middleNum, int Y_coord) {
    int leftSize = middleNum - X_coord + 1;
    int rightSize = Y_coord - middleNum;

    int (*firstCoordinate)[2] =
        new int[leftSize][2];

    int (*comparingCoordinate)[2] =
        new int[rightSize][2];

    for (int i = 0; i < leftSize; i++) {
        firstCoordinate[i][0] =
            coordinateSet[X_coord + i][0];
        firstCoordinate[i][1] =
            coordinateSet[X_coord + i][1];
    }

    for (int j = 0; j < rightSize; j++) {
        comparingCoordinate[j][0] =
            coordinateSet[middleNum + 1 + j][0];
        comparingCoordinate[j][1] =
            coordinateSet[middleNum + 1 + j][1];
    }

    int i = 0;
    int j = 0;
    int k = X_coord;

    while (i < leftSize && j < rightSize) {
```

```

        int initialDistance = distanceFormula(
            firstCoordinate[i][0],
            firstCoordinate[i][1]);

        int comparisonDistance = distanceFormula(
            comparingCoordinate[j][0],
            comparingCoordinate[j][1]);

        if (initialDistance <= comparisonDistance) {
            coordinateSet[k][0] =
                firstCoordinate[i][0];
            coordinateSet[k][1] =
                firstCoordinate[i][1];

            i++;
        }
        else {
            coordinateSet[k][0] =
                comparingCoordinate[j][0];
            coordinateSet[k][1] =
                comparingCoordinate[j][1];

            j++;
        }
        k++;
    }

    while (i < leftSize) {
        coordinateSet[k][0] = firstCoordinate[i][0];
        coordinateSet[k][1] = firstCoordinate[i][1];

        i++;
        k++;
    }
    while (j < rightSize) {
        coordinateSet[k][0] =
            comparingCoordinate[j][0];
        coordinateSet[k][1] =
            comparingCoordinate[j][1];

        j++;
        k++;
    }

    delete[] firstCoordinate;
    delete[] comparingCoordinate;
}

void sortPoints(int coordinates[][2], int leftDistance, int
    rightDistance) {

```

```

        if (leftDistance < rightDistance) {
            int midpoint = leftDistance + (rightDistance -
                leftDistance) / 2;
            sortPoints( coordinates, leftDistance, midpoint);
            sortPoints(coordinates, midpoint + 1, rightDistance)
                ;
            determinePointDistance(coordinates, leftDistance,
                midpoint, rightDistance);
        }
    }

void kClosestPoints(int coordinates[][2], int arraySize, int
    k) {
    if (arraySize == 0) {
        cout << "[]" << endl;
        return;
    }
    for (int i = 0; i < k; i++) {
        cout << "[" << coordinates[i][0]
            << ", " << coordinates[i][1] << "];"
    }
    cout << endl;
}

```

6.7 Testing

Test Case 1 (Single Coordinate):

Input: [5, 5]

k = 1

Output: [5, 5]

hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

Figure 25: Problem 6 - Test Case 1: Single Coordinate

Test Case 2 (Equidistant Coordinates):

Input: [5, 5] [-5, 5]

k = 2

Output: [5, 5] [-5, 5]

hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

Figure 26: Problem 6 - Test Case 2: Equidistant Coordinates


```
Test Case 3 (Return all Coordinates):  
Input: [3, 5] [-2, 5] [7, 10]  
k = 3  
Output: [-2, 5][3, 5][7, 10]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 27: Problem 6 - Test Case 3: Return all Coordinates

```
Test Case 4 (One Closest Point):  
Input: [3, 5] [-2, 5] [7, 10]  
k = 1  
Output: [-2, 5]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 28: Problem 6 - Test Case 4: One Closest Point

```
Test Case 5 (Empty Set):  
  
Input:  
k = 1  
Output: []
```

Figure 29: Problem 6 - Test Case 5: Empty Set

7 Problem 7: K Most Frequent Values

7.1 Understand the Problem

The goal of this problem is to take an array of integers and an integer k , and return an output of the most frequent k integers. If multiple integers have the same frequency, the order should be from least to greatest. Important edge cases to keep in mind are an empty set, a set with multiple of the same integer, a set with no duplicates, and a set with a mixture of both duplicates and non-duplicates. Assume that the integer k does not exceed the length of the array.

7.2 Design the Solution

One method we can use to solve this problem is to use an unordered map, which basically acts as a method of storing pairs. The pairs would essentially be the numbers appearing in the array and the frequency of its appearance in the array. We would repeat this process until we have gone through the array and acquired a solution. After finding the frequency of every integer, we would compare the frequencies to determine the hierarchy.

7.3 Pseudocode

Algorithm 7 K Most Frequent Elements

```
1: function COMPAREFREQUENCIES(x, y, IntegerFrequency)
2:   if IntegerFrequency of x is not equal to IntegerFrequency of y then
3:     return IntegerFrequency of x is greater than IntegerFrequency of y
4:   end if return x is less than y
5: end function
6: function FREQUENCYTRACKER(array, k)
7:   initialize vector frequencyValues
8:   initialize vector finalSet
9:   frequency  $\leftarrow$  empty map
10:  for each number in array do
11:    Increase [number] by 1
12:  end for
13:  for numberPair and IntegerFrequency do
14:    pushback(numberPair.first)
15:  end for
16:  Call Sort values using the following comparison:
17:    Higher frequency comes first
18:    If frequencies are equal, smaller number comes first
19:  for  $i \leftarrow 0$  to  $k - 1$  do
20:    Add frequencyValues[ $i$ ] to finalSet
21:  end for
22: end function
23: function RETURNARRAY(mode)
24:  for each element in mode do
25:    display element
26:  end for
27: end function
```

7.4 Correctness Explanation

This method is correct because it is able to keep track of the pairs stored in the unordered map. The integers with the highest frequency will be placed first, and the integers with the lowest frequency will be placed last. If two integers have the same frequency, the smaller one will be placed first.

7.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log k)$

Average Case: $O(n \log k)$

Best Case: $O(n \log k)$

Space: $O(k)$

Scanning through the array: $O(n)$

7.6 C++ Implementation

```
bool compareFrequencies(
    int x, int y,
    const unordered_map<int, int>& IntegerFrequency) {

    if (IntegerFrequency.at(x) != IntegerFrequency.at(y)) {
        return IntegerFrequency.at(x) > IntegerFrequency.at(
            y);
    }

    return x < y;
}

vector<int> frequencyTracker(vector<int> array, int k) {
    unordered_map<int, int> IntegerFrequency;

    vector<int> frequencyValues;
    vector<int> finalSet;

    for (int i = 0; i < array.size(); i++) {
        IntegerFrequency[array[i]]++;
    }

    for (auto numberPair : IntegerFrequency) {
        frequencyValues.push_back(numberPair.first);
    }

    sort(frequencyValues.begin(), frequencyValues.end(), [&
        IntegerFrequency](int a, int b) {
        return compareFrequencies(a, b, IntegerFrequency);
    });

    for (int i = 0; i < k && i < frequencyValues.size(); i
        ++){

        finalSet.push_back(frequencyValues[i]);
    }
    return finalSet;
}

void returnArray(vector<int> mode) {
    cout << "[";

    for (int i = 0; i < mode.size(); i++) {
```

```

        cout << mode[i];

        if (i < mode.size() - 1) {
            cout << ", ";
        }
    }

    cout << "]" << endl;
}

```

7.7 Testing

```

Test Case 1 (Empty Set):

Input: []
k = 2
Output: []
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

```

Figure 30: Problem 7 - Test Case 1: Empty Set

```

Test Case 2 (Only one integer):

Input: [3, 3, 3, 3, 3]
k = 2
Output: [3]
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

```

Figure 31: Problem 7 - Test Case 2: Only One Integer

```

Test Case 3 (No Duplicating Numbers):

Input: [1, 2, 3, 4, 5]
k = 2
Output: [1, 2]
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

```

Figure 32: Problem 7 - Test Case 3: No Duplicating Numbers

```
Test Case 5 (A Mix of Duplicating and non-Duplicating Numbers):  
Input: [1, 2, 2, 4, 4]  
k = 2  
Output: [2, 4]  
hamzaghosseh@Hamzas-MacBook-Air EECE7205 %
```

Figure 33: Problem 7 - Test Case 4: A Mix of Duplicating and Non-Duplicating Numbers

8 Problem 8: Merge K Sorted Arrays

8.1 Understand the Problem

The objective of this problem is to sort through multiple arrays in one larger array, and essentially combine everything into one array in an order from least to greatest. There is no necessity of repeated sorting since k is the total number of sorted arrays. The edge cases to keep in mind are of course an empty set, all values of the same integer, some empty sets and some filled with multiple values, and a mixture of repeating and non-repeating integers. Assume that every input is a positive integer, and that every array in the input is already sorted from least to greatest.

8.2 Design the Solution

One way to design this algorithm is to use a minimum heap, which entails each input array is already sorted and the smallest element in each sub array can be inserted into the heap. After removing it, the next value in the input arrays comes in and takes the preceding value's spot. This process repeats until all values in each sub-array are organized in order into one large array.

8.3 Pseudocode

Algorithm 8 Merge K Sorted Arrays

```
1: add struct InspectingHeaps{key, arrayPriority, elementPriority}
2: add struct ComparingHeaps
3: return first.key greater than second.key
4: function MERGINGARRAYS(sub-arrays)
5:   heap  $\leftarrow$  empty min-heap
6:   vector in output  $\leftarrow$  empty array
7:   for each sub-array i do
8:     if array i is not empty then
9:       InspectingHeaps myIndex
10:      myIndex.key = first element of array
11:      myIndex.arrayPriority = i
12:      myIndex.elementPriority = 0
13:      push myIndex into minVal
14:      Insert (sub - arrays[i][0], i, 0) into heap
15:     end if
16:   end for
17:   while minVal is not empty do
18:     smallestElement = call top of minVal to locate
19:     pop smallestElement from minVal
20:     Add smallestElement.key to output
21:     nextIndex  $\leftarrow$  elementIndex + 1
22:     if nextIndex is within sub - arrays[arrayIndex] then
23:       Insert (sub - arrays[], myIndex, nextPosition) into heap
24:       Call InsepctingHeaps struct nextElement
25:       nextElement.key = arrayCollection[nextIndex]
26:       nextElement.arrayPriority
27:     end if
28:   end while
29:   return output
30: end function
```

8.4 Correctness Explanation

This method is correct because there is no need for sorting after combining everything into one giant array once you look at every sub-array. And the heap method works with that because it looks at all of the sub-arrays within the larger input array and selectively chooses the smallest value in each of them and puts it into one output without having to sort everything.

8.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log k)$
 Average Case: $O(n \log k)$
 Best Case: $O(n \log k)$
 Space: $O(n + k)$
 Scanning through all elements: $O(n)$

8.6 C++ Implementation

```

struct InspectingHeaps {
    int key;
    int arrayPriority;
    int elementPriority;
};

struct ComparingHeaps {
    bool operator()(InspectingHeaps first, InspectingHeaps
second) {
        return first.key > second.key;
    }
};

vector<int> mergingArrays(vector<vector<int>>
arrayCollection) {
    priority_queue<InspectingHeaps, vector<InspectingHeaps>,
ComparingHeaps> minVal;

    vector<int> output;

    for (int i = 0; i < arrayCollection.size(); i++) {
        if (!arrayCollection[i].empty()) {
            InspectingHeaps myIndex;
            myIndex.key = arrayCollection[i][0];
            myIndex.arrayPriority = i;
            myIndex.elementPriority = 0;
            minVal.push(myIndex);
        }
    }

    while (!minVal.empty()) {
        InspectingHeaps smallestElement = minVal.top();

        minVal.pop();
        output.push_back(smallestElement.key);
        int nextNum = smallestElement.elementPriority + 1;

        if (nextNum < arrayCollection[smallestElement.
arrayPriority].size()) {

            InspectingHeaps nextVal;

```

```

        nextVal.key = arrayCollection[ smallestElement.
            arrayPriority] [nextNum];

        nextVal.arrayPriority = smallestElement.
            arrayPriority;
        nextVal.elementPriority = nextNum;
        minVal.push(nextVal);
    }
}

return output;
}

```

8.7 Testing

```

Test Case 1 (Normal Sorted Arrays):

Input:
[2, 5, 12]
[4, 51, 99]
[23, 65, 95]
Output: [2, 4, 5, 12, 23, 51, 65, 95, 99]
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

```

Figure 34: Problem 8 - Test Case 1: Normal Sorted Arrays

```

Test Case 2 (Different Sized Arrays):

Input:
[12, 51, 121]
[21, 44]
[9]
Output: [9, 12, 21, 44, 51, 121]
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

```

Figure 35: Problem 8 - Test Case 2: Different Sized Arrays

```
Test Case 3 (Duplicate Values):  
  
Input:  
[12, 12, 12]  
[2, 4]  
[3]  
Output: [2, 3, 4, 12, 12, 12]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 36: Problem 8 - Test Case 3: Duplicate Values

```
Test Case 4 (Empty Sub-Array ):  
  
Input:  
[]  
[5, 33]  
[32]  
Output: [5, 32, 33]  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 37: Problem 8 - Test Case 4: Empty Sub-Array

```
Test Case 5 (Empty Input):  
  
Input:  
Output: []  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 38: Problem 8 - Test Case 5: Empty Input

9 Problem 9: Kth Largest Element

9.1 Understand the Problem

The objective of this problem is to take an input array and an integer k , reorganize it so it's from greatest to least, and then return the value in the array that corresponds to the integer k . Important edge cases would be an single element array, an array with an integer duplicate, a standard array of repeating and non-repeating integers, an array where k equals 1, and an array where k equals the size. Assume that k is smaller than the size of the array.

9.2 Design the Solution

Similarly to Problem 8, we can use a min-heap to implement our function. The function will go through the input array while maintaining the heap to not exceed the value of k . Each value will be added to the heap, and if the size of the heap becomes greater than k , the smallest value will be removed. After every value in the input has been iterated through, the heap will contain the k largest elements from the array. At the end, just a single value at the top of min-heap will be returned.

9.3 Pseudocode

Algorithm 9 Kth Largest Element

```
1: function LARGESTK(emptyVector, k)
2:   if  $k \leq 0$  or  $k > \text{size of emptyVector}$  then
3:     return  $-1$ 
4:   end if
5:    $\text{minimum} \leftarrow \text{empty min-heap}$ 
6:   for each  $\text{number}$  in array do
7:     Insert  $\text{number}$  into  $\text{minimum}$ 
8:     if size of  $\text{minimum} > k$  then
9:       call pop  $\text{minimum}$ 
10:    end if
11:  end for
12:  return top element in  $\text{minimum}$ 
13: end function
```

9.4 Correctness Explanation

This function is correct because it implements a min-heap that contains at most k elements. As each value in the array is added to the heap, the smallest value is removed whenever the heap contains more than k elements. This secures that only the k largest elements stay in the heap. Once every element has been processed, the smallest integer remaining in the heap is therefore the k th largest element in the original array.

9.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log k)$

Average Case: $O(n \log k)$

Best Case: $O(n \log k)$

Space: $O(k)$

Scanning through the array: $O(n)$

9.6 C++ Implementation

```
int largestK(vector<int> emptyVector, int k) {
    if (k <= 0 || k > emptyVector.size()) {
        return -1;
    }

    priority_queue<int, vector<int>, greater<int>> minimum;

    for (int i = 0; i < emptyVector.size(); i++) {
        minimum.push(emptyVector[i]);
        if (minimum.size() > k) {
            minimum.pop();
        }
    }
    return minimum.top();
}
```

9.7 Testing

Test Case 1 (Standard Array):

Input: [4, 7, 8, 2, 1]

k = 2

Output: 7

hamzaghosheh@Hamzas-MacBook-Air ECE7205 %

Figure 39: Problem 9 - Test Case 1: Standard Array

```
Test Case 2 (Largest Element):  
Input: [5, 27, 21, 55, 12]  
k = 1  
Output: 55  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 40: Problem 9 - Test Case 2: Largest Element

```
Test Case 3 (Duplicate Elements):  
Input: [88, 88, 2, 43, 6]  
k = 2  
Output: 88  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 41: Problem 9 - Test Case 3: Duplicate Elements

```
Test Case 4 (k equals array size):  
Input: [96, 19, 22]  
k = 3  
Output: 19  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 42: Problem 9 - Test Case 4: k Equals Array Size

```
Test Case 5 (Single Element Array):  
Input: [12]  
k = 1  
Output: 12  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 43: Problem 9 - Test Case 5: Single Element Array

10 Problem 10: Count Inversions

10.1 Understand the Problem

The issue with this problem is to use an inversion, which is a pair of integers where the larger integer comes first before the smaller number. The way of setting this up can come in an array, where we will iterate through it to find any inversion that exists. The output will be the quantity of inversion pairs in the form of an integer. What is unique about this problem is to use a 64-bit integer for the inversion count, which proves to be successful when storing larger numbers compared to a time complexity of $O(n^2)$.

10.2 Design the Solution

One way of designing that works is using merge sort, which is similar to what we've used before in previous problems. Essentially, we will divide the array into multiple sub-arrays to then be merged together back in order. During the process, if an integer in the right sub-array is larger than the integer in the left sub-array, we would establish that that pair is an inversion. This process repeats until all inversion are stored into a 64-bit inversion counter and until the program outputs the total quantity of inversion pairs.

10.3 Pseudocode

Algorithm 10 Count Inversions Using Merge Sort

```

1: function MERGEARRAYS(array, leftHalf, midpoint, rightHalf)
2:   Create copies of the left and right sub-arrays
3:    $m \leftarrow 0$ 
4:    $n \leftarrow 0$ 
5:    $quantity \leftarrow 0$ 
6:   while both sub-arrays have remaining elements do
7:     if  $leftArray[m] \leq rightArray[j]$  then
8:       Add  $leftArray[a]$  to merged array
9:        $m \leftarrow m + 1$  iteration occurs
10:    else
11:      Add  $rightArray[n]$  to the merged array
12:       $totalQuantity \leftarrow quantity + (\text{number of integers remaining in}$ 
    leftArray)
13:       $n \leftarrow n + 1$  second iteration
14:    end if
15:  end while
16:  while  $m$  is less than focus.size do
17:    increment the current element
18:  end while
19:  while  $n$  is less than comparedArray.size do
20:    increment the current element
21:  end while
22:  Add any remaining elements to the merged array
23:  return  $quantity$ 
24: end function
25: function COUNTER(array, leftHalf, rightHalf)
26:   if  $leftHalf \geq rightHalf$  then
27:     return 0
28:   end if
29:    $middle \leftarrow (leftHalf + rightHalf)/2$ 
30:    $quantity \leftarrow \text{INVERSIONQUANTITY}(\text{array}, \text{leftHalf}, \text{quotient})$ 
31:    $quantity \leftarrow quantity + \text{INVERSIONQUANTITY}(\text{array}, \text{quotient} + 1, \text{right-}$ 
    Val)
32:    $quantity \leftarrow quantity + \text{MERGEARRAYS}(\text{array}, \text{leftHalf}, \text{quotient}, \text{right-}$ 
    Val)
33:   return  $quantity$ 
34: end function

```

10.4 Correctness Explanation

This method is correct because it properly uses the bit-integer tracker without having to go with a larger time complexity. When a value from the current

array is smaller than the array you are comparing it with, an inversion forms with the current and remaining values in the array we started with. Plus it is easy to sort when using the 64-bit inversion counter. And this process repeats until all arrays have been merged and until we have a final count for inversions.

10.5 Complexity Analysis

The time complexity for this problem goes as follows:

Worst Case: $O(n \log n)$

Average Case: $O(n \log n)$

Best Case: $O(n \log n)$

Space: $O(n)$

Scanning through the array: $O(n)$

10.6 C++ Implementation

```
long long mergeArrays(vector<int>& array, int leftHalf, int
    midpoint, int rightHalf) {

    vector<int> focus;
    vector<int> comparedArray;

    for (int i = leftHalf; i <= midpoint; i++) {
        focus.push_back(array[i]);
    }

    for (int i = midpoint + 1; i <= rightHalf; i++) {
        comparedArray.push_back(array[i]);
    }

    int m = 0;
    int n = 0;
    int current_element = leftHalf;
    long long quantity = 0;

    while (m < focus.size() && n < comparedArray.size()) {
        if (focus[m] <= comparedArray[n]) {
            array[current_element] = focus[m];
            m++;
        }
        else {
            array[current_element] = comparedArray[n];
            quantity += focus.size() - m;
            n++;
        }
        current_element++;
    }
}
```

```

        while (m < focus.size()) {
            array[current_element] = focus[m];
            m++;
            current_element++;
        }

        while (n < comparedArray.size()) {
            array[current_element] = comparedArray[n];
            n++;
            current_element++;
        }
        return quantity;
    }

long long counter(vector<int>& array, int leftHalf, int
rightHalf) {
    if (leftHalf >= rightHalf) {
        return 0;
    }

    int quotient = leftHalf + (rightHalf - leftHalf) / 2;
    long long quantity = 0;
    quantity += counter(array, leftHalf, quotient);
    quantity += counter(array, quotient + 1, rightHalf);
    quantity += mergeArrays(array, leftHalf, quotient,
        rightHalf);

    return quantity;
}

```

10.7 Testing

Test Case 1 (Example Input):

Input: [2, 4, 1, 3, 5]

Output: Inversion Count 3

hamzaghosheh@Hamzas-MacBook-Air EECE7205 %

Figure 44: Problem 10 - Test Case 1: Example Input

```
Test Case 2 (Given Sorted Array):  
  
Input: [10, 11, 12, 13, 14]  
Output: Inversion Count 0  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 45: Problem 10 - Test Case 2: Given Sorted Array

```
Test Case 3 (Reversed Order):  
  
Input: [9, 8, 7, 6]  
Output: Inversion Count 6  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 46: Problem 10 - Test Case 3: Reversed Order

```
Test Case 4 (Duplicate Values):  
  
Input: [12, 12, 12, 12, 12]  
Output: Inversion Count 0  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 47: Problem 10 - Test Case 4: Duplicate Values

```
Test Case 5 (Empty Array):  
  
Input: []  
Output: Inversion Count 0  
hamzaghosheh@Hamzas-MacBook-Air EECE7205 %
```

Figure 48: Problem 10 - Test Case 5: Empty Array

11 References

All references and external resources used in writing the C++ implementation of each problem are cited directly within the original C++ source code. Please see the submitted C++ file for the corresponding links and citations on GitHub for your reference.

Problem 1

- Line 14: <https://chatgpt.com/c/6ab56fce-8a20-83ea-8e9b-befbbd1eabe4>
- Line 26: <https://chatgpt.com/c/6ab56f64-2a14-83ea-9af3-e95aaaded364>
- Line 54: <https://www.geeksforgeeks.org/cpp/delete-in-c/>

Problem 2

- Line 134: <https://chatgpt.com/c/6ab56e9a-9f04-83ea-a913-17263341ff8e>
- Line 150: <https://www.geeksforgeeks.org/cpp/swap-in-cpp/>

Problem 3

- Line 178: https://en.cppreference.com/cpp/string/basic_string_to_string
- Line 182: https://www.w3schools.com/cpp/ref_algorithm_sort.asp

Problem 4

- Line 200: <https://chatgpt.com/c/6ab569d5-b8b4-83ea-8d96-5983d559b915>
- Line 231: <https://chatgpt.com/c/6ab56bf2-e6cc-83ea-a1e5-1bf8d050763a>

Problem 5

- Line 278: <https://chatgpt.com/c/6ab5928e-ea88-83ea-b570-47205e73d2c5>

Problem 6

- Line 382: <https://chatgpt.com/c/6ab59063-5dc4-83e9-b436-129150e5e5de>

Problem 7

- Line 420: <https://stackoverflow.com/questions/4629317/what-does-int-mean>
- Line 430: <https://chatgpt.com/c/6ab563e3-0be8-83ea-a948-dffd8aebe066>
- Line 439: <https://stackoverflow.com/questions/6900459/the-new-keyword-auto-when-should-it-be-used-to-declare-a-variable-type>
- Line 441: <https://chatgpt.com/c/6ab562ec-e590-83ea-b979-827956629f0d>
- Line 447: <https://chatgpt.com/c/6ab57d32-2b58-83ea-bf52-92a95fcec237>
- Line 451: <https://www.geeksforgeeks.org/cpp/vector-push-back-cpp-stl/>

Problem 8

- Line 476: <https://chatgpt.com/c/6ab55bd5-874c-83ea-bc29-e763cf1a7f51>
- Line 486: <https://www.geeksforgeeks.org/cpp/priority-queue-in-cpp-stl/>
- Line 502: <https://chatgpt.com/c/6ab57eb5-d190-83ea-8a1b-d90a6a9637c2>

Problem 9

- Line 523: <https://www.geeksforgeeks.org/cpp/array-size-c-stl/>
- Line 529: <https://www.geeksforgeeks.org/cpp/priority-queue-in-cpp-stl/>

Problem 10

- Line 597: <https://www.geeksforgeeks.org/cpp/difference-between-long-int-and-long-long-int-in-c-cpp/>